Helm Gateway & Certificate Patterns

Why This Matters

- Stardog deployments can reuse a shared Envoy Gateway or create their own.
- Cert-manager automation may live in Terraform or inside the Helm chart.
- Pick the model that matches your platform’s security boundaries.

Pattern #1 – Certificate Outside, Routes Inside

- **How it works:** Terraform (or another platform IaC) installs cert-manager, ACME Issuer, Certificates,
  the shared Gateway listeners. Each Stardog release only renders `HTTPRoute`/`TCPRoute` resources
  via `parentRefs`.
- **Benefits:**
- Centralized TLS secrets and ACME accounts; no elevated permissions for app teams.
- A single Let’s Encrypt certificate (e.g., `profile-01e.sd-testlab.com`) serves both HTTPS and BI TCP (`T
  listener on port 5806).
- Shared Gateway enforces consistent listeners (`http`, `https`, `bi-tcp`) and DNS coverage (`sparql.*`, `l
  `launchpad.*`).
- **Watchouts:**
- Helm values must set `gateway.http.createGateway=false` and `gateway.http.tls.enabled=false`; TLS p
  come entirely from the platform.
- Terraform must keep the Issuer + Certificate manifests up to date (`envs/.../ingress.tf` handles this in o
  repo).

Pattern #2 – Everything Outside the Chart

- **How it works:** Routes, Gateways, Certificates, and LoadBalancers are all defined in a central IaC re
  Helm charts expose Services only.
- **Benefits:**
- Strong separation of duties and auditability; ideal for regulated environments or strict GitOps flows.
- Zero Kubernetes RBAC surface granted to app releases.
- **Trade-offs:**
- Slower iteration—every hostname/path change requires a platform pull request.
- Helm values never reference `gateway.*`, so documentation must explain how traffic arrives at the Ser

Pattern #3 – Route + Certificate Inside the Chart

- **How it works:** Set `gateway.http.createGateway=true`, enable `gateway.http.tls`, and optionally `gateway.tcpBi.tls`. The chart renders the Gateway, listeners, Certificates, and routes in one release.
- **Benefits:**
- Self-contained; easy to reason about, great for isolated dev/test clusters or single-tenant environments
- Lets teams experiment with new Gateway features (redirects, launchpad proxy, BI TCPRoute) without platform changes.
- **Trade-offs:**
- Multiple releases may each create their own Gateway + LoadBalancer, consuming IPs and duplicating certificates.
- Requires the Helm release to have `cluster-admin`-like permissions to install CRDs, Gateways, and Se

Choosing the Right Pattern

Question	Preferred Pattern
Do you operate a shared Envoy Gateway managed by Terraform?	**Pattern #1**
Do security teams manage every external listener and cert?	**Pattern #2**
Are you deploying to isolated clusters or need app-level autonomy?	**Pattern #3**

> Tip: Regardless of pattern, remember that Gateway listeners own the external port. Only one listener (
or chart-managed) can terminate TLS for a given IP:port combination.

Hands-On Checklist (Pattern #1)

1. Confirm Terraform applied the ACME Issuer & Certificate (`null_resource.envoy_acme_issuer` / `envoy_certificate`).
2. In `values.stardog.yaml`, set:

- `gateway.http.domain`, `createGateway=false`, `tls.enabled=false`
- `gateway.tcpBi.parentRefs` → shared `bi-tcp` listener (no TLS block)

3. Enable Launchpad redirect if needed:

- `gateway.http.redirect.enabled=true`

4. Deploy Helm; verify routes attach via `kubectl get httproute`.
5. Validate TLS with `openssl s_client -connect bi.<domain>:5806`.

Key Takeaways

- Certificates and routes can live in different layers; just ensure ownership is clear.
- The Stardog chart happily acts as a “route only” consumer when a shared Gateway handles TLS.
- For BI TCP TLS, either rely on the shared listener (Pattern #1/#2) or opt into `createGateway=true` (Pa
  #3).
- Document your chosen pattern so every environment (profile-01e, profile-01c, etc.) stays consistent.